

# Automatic Generation of System Test Cases from Use Case Specifications

Chunhui Wang<sup>†</sup>, Fabrizio Pastore<sup>‡</sup>, Arda Goknil<sup>†</sup>, Lionel Briand<sup>†</sup>, Zohaib Iqbal<sup>†‡</sup>

<sup>†</sup> Interdisciplinary Centre for Security, Reliability and Trust, University of Luxembourg, Luxembourg

<sup>‡</sup> Quest Lab, National University of Computer & Emerging Sciences (FAST NU), Islamabad, Pakistan  
{chunhui.wang,fabrizio.pastore,arda.goknil,lionel.briand}@uni.lu zohaib.iqbal@nu.edu.pk

## ABSTRACT

In safety critical domains, system test cases are often derived from functional requirements in natural language (NL) and traceability between requirements and their corresponding test cases is usually mandatory. The definition of test cases is therefore time-consuming and error prone, especially so given the quickly rising complexity of embedded systems in many critical domains. Though considerable research has been devoted to automatic generation of system test cases from NL requirements, most of the proposed approaches require significant manual intervention or additional, complex behavioral modelling. This significantly hinders their applicability in practice.

In this paper, we propose Use Case Modelling for System Tests Generation (UMTG), an approach that automatically generates executable system test cases from use case specifications and a domain model, the latter including a class diagram and constraints. Our rationale and motivation are that, in many environments, including that of our industry partner in the reported case study, both use case specifications and domain modelling are common and accepted practice, whereas behavioural modelling is considered a difficult and expensive exercise if it is to be complete and precise. In order to extract behavioral information from use cases and enable test automation, UMTG employs Natural Language Processing (NLP), a restricted form of use case specifications, and constraint solving.

## Categories and Subject Descriptors

D.2.5 [Software Engineering]: Testing and Debugging

## General Terms

Reliability, Verification

## Keywords

NLP, Use Case Specifications, Test Cases Generation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [Permissions@acm.org](mailto:Permissions@acm.org).

ISSTA '15, July 13–17, 2015, Baltimore, MD, USA  
© 2015 ACM. 978-1-4503-3620-8/15/07...\$15.00  
<http://dx.doi.org/10.1145/2771783.2771812>

## 1. INTRODUCTION

The complexity of embedded software in safety critical domains, e.g. automotive, has significantly increased over the years. System test cases in these domains are often manually derived from functional requirements in natural language (NL). One important motivation is to ensure clear traceability between requirements and system test cases. As a result, the definition of test cases is time-consuming and challenging under time constraints. In this context, automatic test generation not only reduces the cost of testing but also helps guarantee that test cases properly cover all requirements, a very important objective in safety critical systems and the standards they need to comply with.

The benefits of automatic test generation are widely acknowledged today and there are many proposed approaches in the literature [10]. Many approaches [26] require that system specifications be captured as UML behavioural models such as activity diagrams [19], statecharts [24], and sequence diagrams [21]. In modern industrial systems, these behavioural models tend to be complex and expensive if they are to be precise and complete enough to support test automation, and are thus often not part of development practice. There is work [31] [29] [30] that generates test models (UML) from NL requirements, but these generated models need to be manually edited to enable test automation, thus again creating scalability issues. In approaches generating test cases directly from NL requirements [32] [25] [12], test cases are not executable and need significant manual intervention, especially regarding test inputs and outputs. In contrast, our goal in this paper is to enable fully automated test generation from NL requirements, with no behavioural modelling and minimal domain modelling. Our motivation is to rely, to the largest extent possible, on practices that are already in place in many environments developing embedded systems, including the industry partner with whom we performed the case study reported in this paper. Use case specifications are widely used for communicating requirements among stakeholders and, in particular, facilitating communication with customers. Domain modelling is a common way to clarify the terminology and concepts shared among all stakeholders and thus avoid misunderstandings.

In this paper, we propose *Use Case Modelling for System Tests Generation (UMTG)*, an approach that aims at generating executable system test cases by exploiting the behavioural information implicitly described in use case specifications. UMTG requires a domain model of the system, which enables the definition of constraints that are used by UMTG to generate test data and oracles. Use case speci-

cations and domain models are already common in requirements engineering practice [18], such as the partner’s organisation in our case study. Consistent with the objectives stated above, we avoid behavioral modelling (e.g. activity and sequence diagrams) by applying Natural Language Processing (NLP) to a more structured and analysable form of use case specifications, i.e., Restricted Use Case Modeling (RUCM) [31]. RUCM is based on a template with restriction rules, allowing the extraction of behavioral information by reducing imprecision and incompleteness in use cases. RUCM was previously evaluated through controlled experiments and has shown to be usable and beneficial with respect to making use case specifications less ambiguous and more amenable to precise analysis and design [31]. In addition, to generate test data via constraint solving [3], UMTG expects constraints, referring to the domain model, to be defined with the Object Constraint Language (OCL) [2] since OCL is the natural choice when defining high-level constraints on class diagrams.

Our approach, UMTG, employs NLP to build *Use Case Test Models (UCTMs)* from RUCM specifications. A UCTM captures the control flow implicitly described in the RUCM specification. During NLP, a list of textual descriptions of pre, post and guard conditions in use cases is extracted. The software engineer further manually reformulates these textual descriptions using OCL constraints based on the domain model, iteratively refining the latter when required. Our approach combines UCTMs with the OCL constraints to enable automated test generation. As a final step, it automatically generates system test cases with test inputs from UCTMs via constraint solving [3]. Test oracles are generated by processing postconditions. To summarise, the contributions of this paper are:

- UMTG, an approach for the automatic generation of executable system test cases from use case specifications and a domain model, without resorting to behavioral modelling;
- an NLP technique generating test models (UCTMs) from use case specifications expressed with RUCM;
- an algorithm combining UCTMs and constraint solving to automatically generate test inputs;
- an industrial case study in the automotive domain demonstrating in a realistic context the feasibility of the approach.

This paper focuses on automation and technological aspects of our approach and is structured as follows. Section 2 introduces the industrial context for which we developed UMTG, in order to provide concrete motivations. In Section 3, we provide an overview of UMTG. Section 4 focuses on methodological remarks regarding the use of RUCM to elicit requirements while Section 5 presents the NLP pipeline in our approach. In Sections 6 to 10, we detail the automated activities in UMTG. In Section 11, we present our empirical evaluation. Section 12 discusses related work. We conclude the paper in Section 13.

## 2. MOTIVATION AND CONTEXT

The context for which we developed UMTG is that of safety-critical embedded software in the automotive domain. Though not in any way unique, automotive software is a representative example of domains for which compliance with

requirements should be demonstrated through documented test cases. For instance, ISO-26262 [15], an automotive standard for functional safety, states that all system requirements should be properly tested by corresponding test cases.

In this paper, we use the system *BodySense<sup>TM</sup>* as a case study and also to motivate and illustrate UMTG. *BodySense* is a safety-critical automotive software developed by IEE [1]. It monitors a car seat via an electrical field sensor to classify the occupant. It disables the airbag for children and unoccupied seats while it ensures airbag deployment for adults. Furthermore it also includes a seat belt reminder function.

Table 1 gives a simplified version of a real test case for a scenario of the use case ‘*Identify initial occupancy status of a seat*’ for *BodySense*. Lines 1, 3, and 5 provide high-level operation descriptions, i.e., informal descriptions of the operations to be performed on the system. These lines are followed by the name of the functions that should be executed by the test driver along with the corresponding input/output values. For instance, Line 4 invokes the function *SetBus* with a value indicating that the test driver should simulate the presence of an adult on the seat (for simplicity assume that the field sensor positioned on a seat sends the value 85 on the bus when an adult is seated).

**Table 1: An example test case for *BodySense<sup>TM</sup>***

| Line | Operation                                                                             | Inputs/Expectations                 |
|------|---------------------------------------------------------------------------------------|-------------------------------------|
| 1    | <i>Reset power and wait</i>                                                           |                                     |
| 2    | ResetPower                                                                            | Time=INIT_TIME                      |
| 3    | <i>Set occupant status - Adult</i>                                                    |                                     |
| 4    | SetBus                                                                                | Channel = RELAY<br>Capacitance = 85 |
| 5    | <i>Check SeatBeltReminder status is Occupied and AirBagControl status is Occupied</i> |                                     |
| 6    | ReadAndCheckBus                                                                       | D0=OCCUPIED<br>D1=OCCUPIED          |

Exhaustive test cases needed to validate a safety-critical embedded system are difficult both to derive and maintain as requirements are changing. For instance, the real test case from which we derived the simplified test case in Table 1, includes 30 test steps, around 50 variable assignments, and 15 references to other specifications providing further explanations about the inputs. The effort required to specify test cases for the whole *BodySense* is overwhelming. A test suite exercising all use case scenarios would include more than 400 such test cases. Without automated test case generation, such testing activity remains not only expensive but also infeasible under typical time constraints.

Within the context of testing safety-critical embedded software such as *BodySense*, we identify three challenges that need to be considered for the automatic generation of system test cases from functional requirements:

**Feasible Modelling.** Most of the existing automatic system test generation approaches are model-based and rely upon behavioural models such as state, sequence or activity diagrams. In complex industrial systems, behavioural models that are precise enough to enable test automation are so complex that their specification cost is prohibitive and the task is often perceived as overwhelming by engineers. To evaluate the applicability of behavioral modelling on *BodySense*, we asked the IEE software engineers to specify system sequence diagrams (SSDs) corresponding to the use cases of *BodySense*. For example, the SSD for the use case ‘*Identify initial occupancy status of a seat*’, includes 74 messages, 19 nested blocks, and 24 references to other SSDs that had

to be derived. This was clearly considered too complex for software engineers and required significant help from the authors of this paper, and many iterations and meetings. Our conclusion is that the adoption of behavioural modelling, at the level of detail required for automated testing, is not a practical option for system test automation except when detailed behavioural models are already used by software engineers for other purposes, e.g. software design.

**Test Data Generation.** Without behavioral modelling, most approaches mainly exploit NL requirements specifications in which it is hard to extract test data for executable test cases. Test cases derived in such a way typically require significant manual intervention. For instance, even the simplified test case in Table 1 has 50 variable assignments to be manually entered as test input. Automatically generating test data, and not just abstract test scenarios, is therefore important.

**Deployment of the Software under Test.** Execution of test cases for a system like *BodySense* entails the deployment of software under test on the target environment. To speed up software testing, test case execution is typically automated through test scripts invoking test driver functions. These functions simulate sensor values and read computational results from a communication bus. Any test generation approach should generate appropriate functions and test data in a processable format for the test driver. For instance, the test drivers in *BodySense* need to invoke driver functions (e.g., *SetBus*) to simulate occupancy on a seat.

### 3. OVERVIEW OF THE APPROACH

Figure 1 shows the main steps of the approach. Our goal in UMTG is to address the challenges given in Section 2. In UMTG, behavioral information and high-level operation descriptions are generated from use cases (*the first challenge*), test inputs are generated through constraint solving (*the second challenge*), while test driver functions corresponding to informal descriptions and oracles implementing the post-conditions of the use case scenario are generated through mapping tables provided by the software engineer (*the third challenge*).

The software engineer elicits requirements with RUCM (Step 1). The domain model is manually created as a UML class diagram (Step 2). UMTG automatically checks if the domain model includes all entities mentioned in the use cases (Step 3) and tool support is provided to guide engineers in completing the domain model. NLP is used to extract domain entities from the use cases. Missing entities are shown to the software engineer who refines the domain model (Step 4). Steps 3 and 4 are iterative: the domain model is refined until it is complete.

Once the domain model is completed, textual descriptions of pre, post and guard conditions in the use cases are automatically extracted (Step 5) to be reformulated as OCL constraints by engineers (Step 6). UMTG further processes the use cases with the OCL constraints to generate a Use Case Test Model for each use case (Step 7). A generated test model is a directed graph that explicitly captures the implicit behavioural information in the corresponding use case.

UMTG relies on constraint solving for OCL constraints that are attached to the nodes of the test models. The goal is to generate test inputs associated with use case scenarios (Step 8). We use the term use case scenario for a sequence of

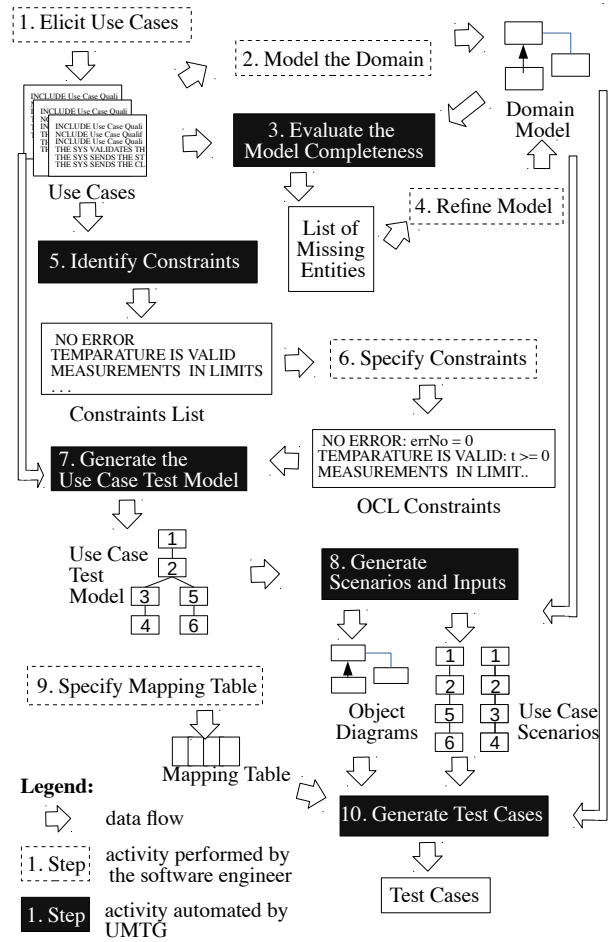


Figure 1: Overview of the UMTG Approach

use case steps that starts with a use case precondition and ends with a postcondition of either a basic or alternative flow. Test inputs cover all paths in the testing model and therefore all possible use case scenarios.

The software engineer provides a mapping table that maps high-level operation descriptions and test inputs to the concrete driver functions and inputs that should be executed by the test case (Step 9). Executable test cases are automatically generated through the mapping table (Step 10). If the test infrastructure and hardware drivers change in the course of the system lifespan, then only the mapping table needs to change.

The rest of the paper provides a detailed description of each step of UMTG shown in Figure 1, with a focus on how we achieved automation.

### 4. REQUIREMENTS ELICITATION

This section briefly introduces RUCM, a use case format that provides restriction rules and specific keywords constraining the use of natural language in use cases. For details, the reader is referred to [31]. The goal of RUCM is to reduce ambiguity and to facilitate automated analysis of use cases. Controlled experiment results show that the restriction rules in RUCM are overall applicable and beneficial [31]. Since RUCM was not originally designed for test generation, we introduce some extensions to RUCM for UMTG.

**Table 2: Use Case *Identify Initial Occupancy Status of a Seat***

|    |                                                                                                                                                                                      |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <b>Precondition</b>                                                                                                                                                                  |
| 2  | The system has been initialized                                                                                                                                                      |
| 3  | <b>1.1 Basic Flow</b>                                                                                                                                                                |
| 4  | 1. The seat SENDS occupancy status TO the system.                                                                                                                                    |
| 5  | 2. INCLUDE USE CASE Classify occupancy status.                                                                                                                                       |
| 6  | 3. The system VALIDATES THAT the occupant class for airbag control is valid and the occupant class for seat belt reminder is valid.                                                  |
| 7  | 4. The system SENDS the occupant class for airbag control TO AirbagControlUnit.                                                                                                      |
| 8  | 5. The system SENDS the occupant class for seat belt reminder TO SeatBeltControlUnit.                                                                                                |
| 9  | Postcondition: The occupant class for airbag control and the occupant class for seat belt reminder have been sent.                                                                   |
| 11 | <b>1.2 Bounded Alternative Flow</b>                                                                                                                                                  |
| 12 | RFS 2-3                                                                                                                                                                              |
| 13 | 1. IF voltage error is detected THEN                                                                                                                                                 |
| 14 | 2. The system resets classification filters.                                                                                                                                         |
| 15 | 3. RESUME STEP 1.                                                                                                                                                                    |
| 16 | 4. ENDIF                                                                                                                                                                             |
| 17 | Postcondition: Classification filters have been reset.                                                                                                                               |
| 18 | <b>1.3 Specific Alternative Flow</b>                                                                                                                                                 |
| 19 | RFS 3                                                                                                                                                                                |
| 20 | 1. IF the occupant class for airbag control is not valid and the occupant class for seat belt reminder is not valid THEN                                                             |
| 21 | 2. The system SENDS the previous occupant class for airbag control TO AirbagControlUnit.                                                                                             |
| 22 | 3. The system SENDS the previous occupant class for seat belt reminder TO SeatBeltControlUnit.                                                                                       |
| 23 | 4. RESUME STEP 1.                                                                                                                                                                    |
| 24 | 5. ENDIF                                                                                                                                                                             |
| 25 | Postcondition: The previous occupant classes for airbag control and seat belt reminder has been sent to AirbagControlUnit and to SeatBeltControlUnit respectively.                   |
| 26 | <b>1.4 Specific Alternative Flow</b>                                                                                                                                                 |
| 27 | RFS 3                                                                                                                                                                                |
| 28 | 1. IF the occupant class for seat belt reminder is not valid THEN                                                                                                                    |
| 29 | 2. The system SENDS the occupant class for airbag control TO AirbagControlUnit.                                                                                                      |
| 30 | 3. The system SENDS the previous occupant class for seat belt reminder TO SeatBeltControlUnit.                                                                                       |
| 31 | 4. RESUME STEP 1.                                                                                                                                                                    |
| 32 | 5. ENDIF                                                                                                                                                                             |
| 33 | Postcondition: The occupant class for airbag control has been sent to AirbagControlUnit and the previous occupant class for seat belt reminder has been sent to SeatBeltControlUnit. |

Table 2 provides a simplified version of the use case ‘*Identify initial occupancy status of a seat*’ of *BodySense* written according to the RUCM rules. We omit some basic information such as actors and dependencies. The use case has one basic and three alternative flows (see Lines 3, 11, 18, 26).

A basic flow describes a main successful path that satisfies stakeholder interests. It contains a sequence of steps and a postcondition (Lines 3-9). A step can be one of the following interactions: an actor sends a request and/or data to the system (Line 4); the system validates a request and/or data (Line 6); the system replies to an actor with a result (Line 7). A step can also capture the system altering its internal state (Line 14). In addition, the inclusion of use cases needs to be specified as steps. This is the case of Line 5, as denoted by the presence of the keyword ‘*INCLUDE USE CASE*’ (some RUCM keywords are written in capital letters for readability).

The keyword ‘*VALIDATES THAT*’ (Line 6) indicates a condition that must hold to proceed to the next step, otherwise an alternative flow is taken. The system proceeds to Step 4 (Line 7) only if the occupant classes for the airbag control and the seat belt reminder are valid (Line 6).

Alternative flows describe other scenarios, both success and failure. An alternative flow always depends on a con-

dition for the basic flow. In RUCM, there are three types of alternative flows: *specific*, *bounded* and *global*. A specific alternative flow refers to a step in the basic flow (Lines 19 and 27). A bounded alternative flow refers to more than one step in the basic flow (Line 12) while a global alternative flow refers to any step in the basic flow. For specific and bounded alternative flows, the keyword ‘*RFS*’ is used to refer to one or more reference steps (Lines 12, 19, and 27).

Bounded and global alternative flows begin with the keyword ‘*IF .. THEN*’ for the guard condition under which the alternative flow is taken (Line 13). Specific alternative flows do not necessarily begin with the keyword ‘*IF .. THEN*’ since a guard condition is already indicated in its reference flow step (Line 6). It is possible to have composite conditions further refined in multiple alternative flows. This is the case for Table 2 in which multiple alternative flows are considered to be executed (Lines 12, 19, and 27) when the condition in the reference flow step (Line 6) evaluates to false. This case is not covered by RUCM. Therefore, UMTG suggests to use the ‘*IF ... THEN*’ keyword also in specific alternative flows (Lines 20 and 28). The alternative flows are evaluated in the order they appear in the use case.

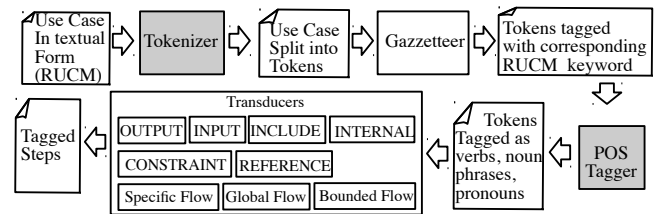
UMTG introduces some other extensions to RUCM regarding the usage of ‘*IF*’ conditions and the way input/output messages are expressed. UMTG follows the guidelines that suggest not to use multiple branches within the same use case path [18], thus enforcing the adoption of ‘*IF*’ conditions only as a means to specify guard conditions for alternative flows.

UMTG introduces the keyword ‘*SENDS ... TO*’ as an RUCM extension for the system-actor interactions. The keyword eases automatic identification of steps for these interactions. According to our experience, in embedded systems the system-actor interactions are always specified in terms of messages. For instance, Step 1 in Table 2 (Line 4) indicates an input message from the seat to the system while Step 4 (Line 7) contains an output message from the system to the airbag. Additional keywords can be defined for other types of systems.

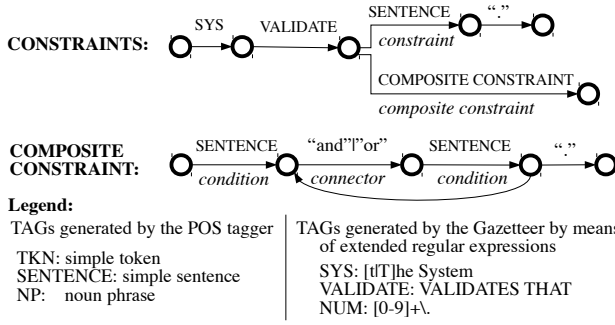
## 5. NLP PIPELINE FOR UMTG

Three UMTG steps in Figure 1, ‘evaluate the model completeness’, ‘identify constraints’, and ‘generate the use case test model’, are supported by an NLP application to extract behavioral information from RUCM use cases.

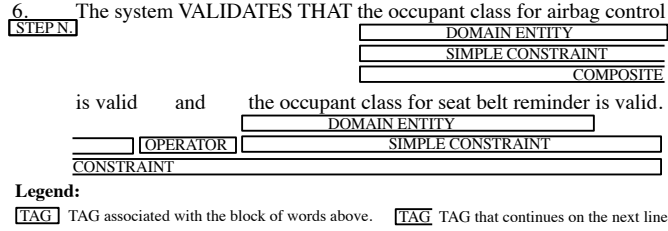
The NLP application in UMTG is based on the GATE workbench [8], an open source NLP framework, and implements the analysis pipeline in Figure 2. The pipeline includes both default NLP components (grey) and components built to process RUCM use cases (white). The *Tokenizer* splits the use cases into tokens. The *Gazetteer* iden-



**Figure 2: NLP pipeline applied to extract the behaviour of a Use Case**



**Figure 3: Part of the transducer that identifies constraints**



**Figure 4: Tags associated with the use case step in Lines 6 of Table 2**

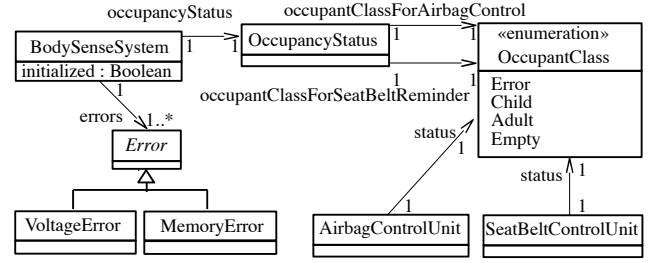
ties the RUCM keywords. The *POS Tagger* tags tokens according to their nature: *verb*, *noun*, and *pronoun*. The pipeline is terminated by a set of *transducers* that tag block of words to distinguish the kinds of RUCM steps: output, input, include, and internal operations. The pipeline has also *transducers* to distinguish specific, bounded and global alternative flows, reference blocks, and constraints part of guard and post conditions.

Figure 3 gives an example transducer for constraints. Capital names on the arrows correspond to the transducer's inputs, i.e., tags previously identified by either the POS tagger, the gazetteer or other transducers. Italic names show the tags assigned by the transducer to the words corresponding to the transducer input. Figure 4 gives the tags associated with the use case step in Line 6 of Table 2 after the execution of the transducer in Figure 3. In Figure 4, multiple tags are assigned to the same block of words: the clause 'the occupant class for airbag control is valid' is tagged both as a *simple constraint* and as a part of a *composite constraint*.

## 6. EVALUATION OF THE MODEL COMPLETENESS

The completeness of the domain model is important to generate correct and complete test inputs. UMTG automatically evaluates the model completeness (Step 3 in Figure 1) by identifying missing domain entities. This is done by checking correspondences between the domain entities identified by the NLP application and the entities in the domain model. Please note that we assume naming conventions for the use cases and domain model<sup>1</sup>.

<sup>1</sup>The precise identification of correspondences between the domain entities in the use cases and the entities in the domain model in the presence of inconsistent naming conven-



**Figure 5: Portion of the domain model for *BodySense*<sup>TM</sup>**

Sometimes domain entities identified in a use case are not modelled as classes but as attributes. Figure 5 shows a simplified portion of the domain model for *BodySense* where the domain entities 'occupant class for airbag control' and 'occupant class for seat belt reminder' are modelled as attributes of the class *OccupancyStatus*. UMTG follows a simple yet effective solution to check class and attribute names. For each domain entity identified through NLP, UMTG generates an entity name by removing all white spaces and by putting all first letters following white spaces in capital. For instance, the domain entity 'occupant class for airbag control' becomes 'OccupantClassForAirbagControl'. UMTG checks generated entity names in the domain model. If the entity name appears either as class name or as an attribute name, the entity is considered present. Otherwise it is considered missing.

## 7. IDENTIFICATION OF CONSTRAINTS

The pre- and guard- conditions used in a use case specification capture the conditions under which a given use case scenario is executed, in other words they allow to determine the inputs that enable the covering of a given use case scenario. UMTG automatically identifies these conditions by means of NLP and asks the software engineer to reformulate them as OCL constraints. Since the execution of a scenario may depend also on how the system modifies its internal state, UMTG asks software engineers to specify with OCL the postconditions of the use case steps that represent internal operations. UMTG uses a constraint solver to solve the provided OCL constraints and automatically generate test inputs (see Section 9).

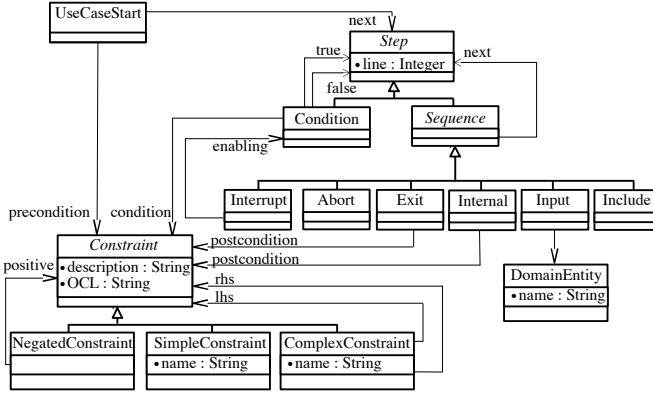
The postconditions of use case flows capture the expected state of the system after the execution of the scenario. UMTG asks the software engineer to reformulate postconditions of the use case flows as OCL constraints. These postconditions are later translated into test oracles (see Section 10).

To minimize manual effort, the NLP application first locates use case conditions and then identifies repeating and negated ones through an NLP transducer. If the use cases both feature a condition and its negation, software engineers are asked to reformulate only the condition as an OCL constraint. The negated OCL version of a constraint is automatically derived. Table 3 provides some of the use case conditions in Table 2 with their corresponding OCL constraints based on the domain model.

tions can be tackled by syntactic and semantic similarity checking tools [23] and is not in the scope of this paper.

**Table 3: Some constraints for the use case ‘Identify Initial Occupancy Status of a Seat’.**

| # | Condition in the Use Case                                                     | Corresponding OCL Constraint                                                                                                                                                                                                                              |
|---|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | the occupant class for airbag control is valid                                | <code>BodySenseSystem.allInstances() → forAll(b b.occupancyStatus.occupantClassForAirbagControl &lt;&gt; OccupantClass :: Error)</code>                                                                                                                   |
| 2 | the occupant class for seat belt reminder is valid                            | <code>BodySenseSystem.allInstances() → forAll(b b.occupancyStatus.occupantClassForSeatBeltReminder &lt;&gt; OccupantClass :: Error)</code>                                                                                                                |
| 3 | voltage error is detected                                                     | <code>Error.allInstances() → select(e e.ocllsKindOf(VoltageError)) → forAll(v v.detected = true)</code>                                                                                                                                                   |
| 4 | the occupant classes for airbag control and seat belt reminder have been sent | <code>AirbagControlUnit.allInstances() → forAll(a a.status = a.bodySense.occupancyStatus.occupantClassForAirbagControl) AND SeatBeltControlUnit.allInstances() → forAll(s s.status = s.bodySense.occupancyStatus.occupantClassForSeatBeltReminder)</code> |
| 5 | The system has been initialized                                               | <code>BodySenseSystem.allInstances() → forAll(b b.initialized = true)</code>                                                                                                                                                                              |
| 6 | The system resets classification filters                                      | <code>BodySenseSystem.allInstances() → forAll(b b.occupancyStatus.occupantClassForSeatBeltReminder = null AND b.occupancyStatus.occupantClassForAirbagControl = null)</code>                                                                              |



**Figure 6: Metamodel for the Use Case Test Model**

## 8. GENERATION OF THE USE CASE TEST MODEL

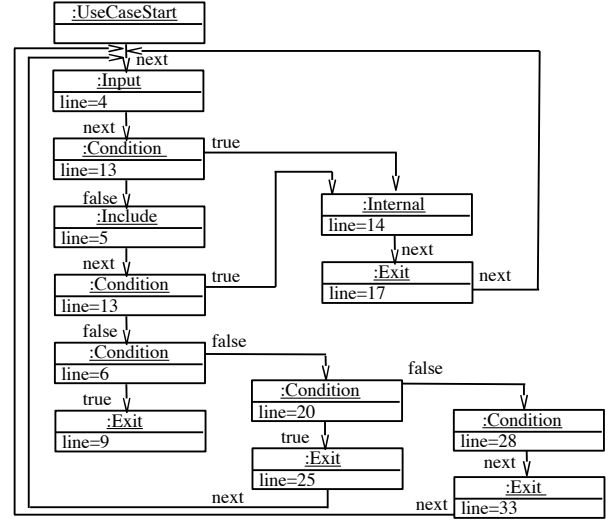
As discussed earlier an RUCM use case along with OCL constraints are used to generate a *Use Case Test Model*. The model makes the implicit control flow in a use case specification explicit, and enables the mapping of use case steps with test case steps. Figure 6 gives the metamodel for the *Use Case Test Model*.

The *UseCaseStart* represents the beginning of a use case and is thus associated with a precondition and to the first step in the use case. *Steps* can be of two different types: *Sequence* and *Condition*. A *Sequence* has only a single successor, while a *Condition* has two possible successors. A *Condition* has a reference to a constraint and it is associated with the steps taken according to the truth value of the constraint.

A *Constraint* has a description informally written in the use case (the second column in Table 3) and a corresponding OCL constraint referring to the domain model (the third column in Table 3). *ComplexConstraints* and *SimpleConstraints* are introduced to reduce the number of manually written OCL constraints. *ComplexConstraints* simply concatenate *SimpleConstraints* using the operators *OR* and *AND*.

An *Input* step indicates that a test case should invoke an input operation during test case execution. *Input* steps reference a *DomainEntity* that represents the data passed as input to the system. An *Interrupt* indicates the presence of inputs that enable the evaluation of a condition of a Global or Bounded Alternative Flow that may disrupt the execution of the basic flow. For this reason, it is linked to the corresponding Condition step.

An *Internal* step indicates that the system alters its internal state. In order to specify the effects of an internal step



**Figure 7: The Use Case Test Model derived from the Use Case in Table 2**

on the system state, the *Internal* step is associated with an OCL constraint specified by the software engineer.

An *Exit* step is the last step of a use case flow and contains a reference to the corresponding postcondition. An *Exit* step for the Basic Flow does not point to any further step, while the *Exit* step of an Alternative Flow points to the step indicated by the keyword *Resume* in the use case. *Abort* steps instead terminate an anomalous execution flow and do not point to any further step.

Figure 7 shows the Use Case Test Model generated for the use case in Table 2 (to improve readability, the figure shows only the objects that model the execution flow).

The Use Case Test Model for a use case is generated by processing the use case specification annotated by the NLP application. UMTG generates a Use Case Test Model for each use case specification. The use case document is parsed from the beginning to the end. Each time a textual element tagged as *Input*, *Include*, *Internal*, or *Conditional* is encountered, the technique generates a corresponding Step instance and connects it to the last Step instance created.

Whenever a domain entity is encountered, a corresponding *DomainEntity* object is created (if it does not exist already) and is linked to the Step that refers to that domain entity.

When a Specific Alternative Flow is found, the *Reference* block is used to find the corresponding Condition step of the Basic Flow and to connect it with the first step of the Alternative Flow. The Condition step of Line 6 in Table 2, for example, is connected with the step of Line 20 (the first of the Specific Alternative Flow).

Global and Bounded Alternative Flows lead to the generation of multiple Condition steps. In UMTG, Global and Bounded Alternative Flows begin with a guard condition, i.e. a Condition step. In case of a Global Alternative Flow, a copy of the Condition step at the beginning of the Alternative Flow is added before all the steps of the basic flow. For a Bounded Alternative Flow, a copy of the Condition step is added before all the steps covered by the Bounded Alternative Flow. In the use case in Table 2, the Condition step of line 13 belongs to a Bounded Alternative Flow that is bounded to Lines 5 to 6, which results in the Use Case Test Model containing two Step instances ‘*Condition line=13*’ before the steps of Lines 5 and 6 (see Figure 7).

If multiple alternative flows depend on the same condition of the basic flow, UMTG simply connects their initial Condition step in cascade following the order in the use case specification. This is the case of the Condition steps in Lines 20 and 28 of the use case in Table 2, which are connected in cascade in Figure 7.

## 9. GENERATION OF SCENARIOS AND TEST INPUTS

Test inputs can be identified from the Use Case Test Model to cover all the possible scenarios of the use case, i.e., all the paths that begin with a *UseCaseStart* step and end with an *Exit* step of the same use case.

According to our definition, a use case scenario does not include the steps in the Use Case Test Model that follow an *Exit* step, and thus we can treat the Use Case Test Model as an acyclic graph. For this reason, we implemented a test generation algorithm that covers all the paths of the Use Case Test Model through a depth first traversal.

Constraint solvers can be used to solve the path condition that must be satisfied to cover a specific path and thus identify test inputs. The constraint solver used by UMTG is dedicated to OCL and relies upon search algorithms to efficiently generate an object diagram with class instances and attribute values that satisfy the given path condition (please refer to [3] for more details).

The following paragraphs describe *GenerateInputs*, the algorithm used by UMTG to build and solve the path conditions. Figure 8 shows the algorithm.

The algorithm takes six inputs: *step*, the step to inspect, *scenario*, the current test scenario, which is a list with the steps covered during the traversal, *pc*, the path condition of the scenario, *results*, a list with the results generated, *dm*, the domain model, and *UCTM*, the Use Case Test Model for which we intend to generate test cases. The algorithm generates a list of pairs  $\langle \text{scenario}, \text{objectDiagram} \rangle$ . Each pair shows an object diagram that includes the input values enabling the coverage of a specific *scenario*.

UMTG generates the scenario and test inputs of the Use Case Test Model under test by invoking *GenerateInputs* using the *UseCaseStart* as the parameter *step*. The lists *scenario*, *pc*, and *results* are initially empty, and are populated by *GenerateInputs* during its execution.

Before executing *GenerateInputs*, UMTG includes into the Use Case Test Model under test the steps of the included use cases. This is done in a fashion that is similar to the generation of interprocedural control flow graphs [13], i.e., by connecting the step that precedes the *Include* step with the *UseCaseStart* of the included Use Case Test Model,

**Require:** *step*, the step to inspect  
**Require:** *scenario*, list of steps that constitute the current test case  
**Require:** *pc*, the current path condition  
**Require:** *results*, list of test inputs already generated  
**Require:** *dm*, the domain model  
**Require:** *M*, the Use Case Test Model under test  
**Ensure:** *results*, a list of pairs  $\langle \text{scenario}, \text{objectDiagram} \rangle$

```

1: if step is UseCaseStart then
2:   scenario  $\leftarrow$  scenario  $\cup$  step
3:   pc  $\leftarrow$  pc AND step.precondition
4:   GenerateInputs(step.next, scenario, pc, tests, dm, M)
5: end if
6: if step is Input then
7:   scenario  $\leftarrow$  scenario  $\cup$  step
8:   GenerateInputs(step.next, scenario, pc, tests, dm, M)
9: end if
10: if step is Condition then
11:   //prepare for visiting the true branch
12:   scenarioT  $\leftarrow$  scenario //create a copy
13:   if step belongs to Global or Bounded then
14:     scenarioT  $\leftarrow$  scenarioT  $\cup$  newInterrupt(step)
15:   end if
16:   scenarioT  $\leftarrow$  scenarioT  $\cup$  step
17:   pcT  $\leftarrow$  pc AND step.condition
18:   GenerateInputs(step.true, scenarioT, pcT, tests, dm, M)
19:   //prepare for visiting the false branch
20:   if step belongs to Global or Bounded then
21:     scenarioF  $\leftarrow$  scenario
22:     pcF  $\leftarrow$  pc
23:   else //step belongs to Specific Alternative Flow
24:     scenarioF  $\leftarrow$  scenario  $\cup$  step
25:     pcF  $\leftarrow$  pc AND ! step.condition
26:   end if
27:   GenerateInputs(step.false, scenarioF, pcF, tests, dm, M)
28: end if
29: if step is Internal then
30:   scenario  $\leftarrow$  scenario  $\cup$  step
31:   pc  $\leftarrow$  pc AND step.postcondition
32:   GenerateInputs(step.next, scenario, pc, tests, dm, M)
33: end if
34: if step is Exit OR Abort then
35:   if step not belongs to M then
36:     GenerateInputs(step.next, scenario, pc, tests, dm, M)
37:   else //step belongs to M
38:     scenario  $\leftarrow$  scenario  $\cup$  step
39:     objectDiagram  $\leftarrow$  oclSolver(pc, dm)
40:     results  $\leftarrow$  results  $\cup$   $\langle \text{scenario}, \text{objectDiagram} \rangle$ 
41:   end if
42: end if

```

**Figure 8: *GenerateInputs*: the Test Generation Algorithm adopted in UMTG**

and by connecting the *Exit* step of the basic flow of the included Use Case Test Model with the step that follows the *Include* step. Additionally, UMTG removes branches that lead to loops in the included Use Case Test Model.

*GenerateInputs* traverses the Use Case Test Model by following a depth first traversal and populates the list *scenario* with the steps visited within a path. Figure 9 shows three scenarios generated for the running example. The following paragraphs describe the activities performed for the different kinds of steps.

In case of a *UseCaseStart* step, *GenerateInputs* adds the pre-condition of the *UseCaseStart* step to the path condition. This is done because it enables the initialisation of the test case (Lines 1 to 5 in Figure 8).

In case of a *Condition* step, *GenerateInputs* visits first the true branch and then the false branch. Visiting the true and the false branch simply consists in recursively invoking *GenerateInputs* after appropriately updating the path condition. When visiting the true branch, *GenerateInputs* adds a new *Interrupt* step to the scenario if the *Condition* step belongs to a Global or Bounded Alternative Flow (Line 14 in Figure 8). This is done because the execution of the true branch of a Bounded or Global Alternative Flow implies the

presence of an interruption. Scenario B in Figure 9 shows the *Interrupt* step added before the *Condition* step of Line 13. The path condition built to visit the true branch of Line 13 is the conjunct of conditions 5 and 3 of Table 3.

Lines 19 to 27 in Figure 8 handle the visit of the false branch. In case the *Condition* step belongs to a Global or Bounded Alternative Flow GenerateInputs does not add the step to the scenario, nor adds the negation of the associated OCL constraints to the path condition (Lines 21 and 22 in Figure 8). This is done because the guard conditions of Global and Bounded Alternative Flows influence the behaviour of a use case only when they are true. The negation of a *Condition* step is added to a scenario only if it belongs to a Specific Alternative Flow (Lines 24, and 25). For example Scenario A in Figure 9, which corresponds to the basic flow of the use case in Table 2, does not include the negation of the condition of the Bounded Alternative flow starting in Line 13 of Table 2. Scenario C (Figure 9) instead includes the negation of the *Condition* step of Line 6, which belongs to a Specific Alternative flow.

An *Internal* step indicates that the system changes its internal state. Since the change may affect the truth value of following Condition steps, GenerateInputs includes in the path condition the constraints associated with the *Internal* steps visited (Line 31 in Figure 8). An example is given by the path condition of Scenario B in Figure 9.

The generation of a scenario terminates when either an *Exit* or an *Abort* step is reached (Line 34 in Figure 8). If the *Exit* step belongs to an included use case, the scenario is not complete yet, and the visit proceeds by invoking GenerateInputs on the following step (Lines 35 to 37). If the *Exit* (or *Abort*) step belongs to the use case under test, GenerateInputs adds the step to the *scenario* (Line 38) and then invokes the OCL solver to solve the current path condition (Line 39). The algorithm then adds to the list *results* a pair  $\langle \text{scenario}, \text{objectDiagram} \rangle$  with the current scenario, and the object diagram returned by the OCL solver (Line 40)<sup>2</sup>.

## 10. GENERATION OF TEST CASES

UMTG generates test cases by processing all the pairs  $\langle \text{scenario}, \text{objectDiagram} \rangle$  produced by the algorithm GenerateInputs. UMTG performs three activities to generate test cases: Identify input values, Generate high-level operation descriptions, and Generate calls to driver functions. These activities are tailored to the specific format described in Section 2 but may be changed, following similar principles, to generate test cases in different formats for embedded systems using a different test infrastructure and hardware.

Table 4 shows a test case automatically generated from the basic flow of the use case in Table 2 (Lines 1,3, 5 are high-level operation descriptions; Lines 2, 4, 6 are driver function calls). This test case corresponds to the manually written test case in Table 1. Figure 10 shows the activities performed by UMTG to automatically generate the test case.

<sup>2</sup>If the path condition is *unsatisfiable* the OCL solver returns *null*, and UMTG will ignore the scenario during test generation because it is infeasible. Infeasible scenarios do not depend on modelling errors but they usually depend on the mutual exclusion of alternative flows of included use cases. UMTG warns for the presence of unreachable steps, i.e. steps not covered by any feasible scenario, which may depend either on modelling errors or limitations of the solver.

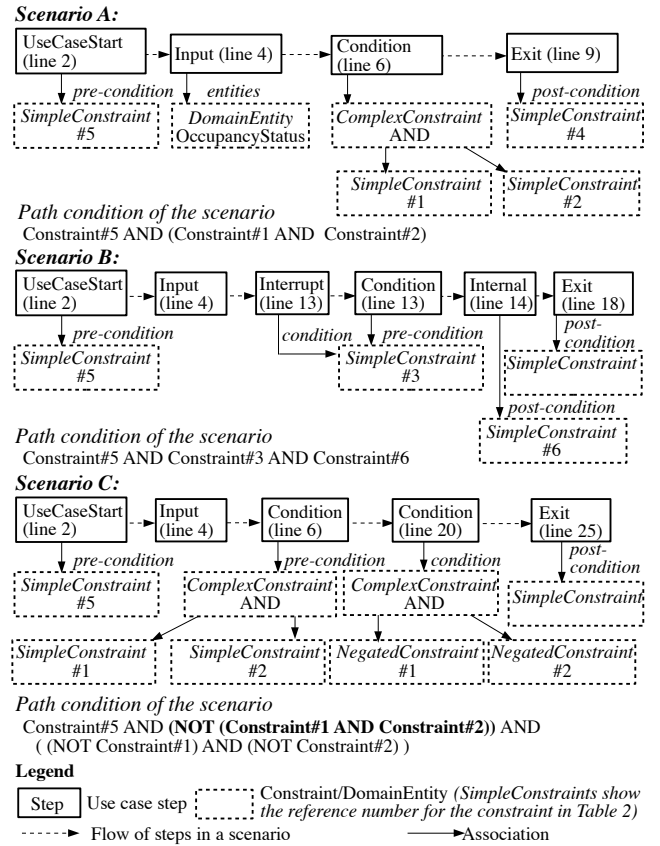


Figure 9: Three scenarios built by UMTG for the use case in Table 2

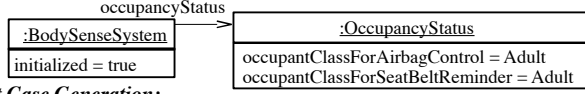
UMTG first processes the object diagram to identify the whole set of input values for the test case (Activity 1 in Figure 10). To this end, UMTG looks for the attributes in the object diagram that appear in the constraints of the scenario.

UMTG generates high-level operation descriptions by first creating *Input* operations and then *Check* operations. UMTG creates a test line with an *Input* operation for every *UseCaseStart*, *Input*, and *Interrupt* step of the scenario. For each input line in the test case, UMTG selects input values that belong to the domain entities appearing in the corresponding step of the scenario. For instance, in Activity 2.2 in Figure 10, the inputs *occupancyStatus.occupantClassForAirbagControl = Adult* and *occupancyStatus.occupantClassForSeatBeltReminder = Adult* are selected when processing the *Input* step in Line 4 (see Scenario A in Figure 9). In fact the *Input* step in Line 4 refers to the domain entity *OccupancyStatus*.

At the end of the test case, UMTG generates a test line with a *Check* operation and the postcondition of the scenario under test (see Activity 2.3 in Figure 10).

To generate calls to driver functions, UMTG parses each high-level operation description by using the mapping table provided by the software engineers (Activity 3 in Figure 10). Table 5 shows a mapping table for *BodySense*. The mapping table is made of four columns. The first two columns provide operation names and regular expressions that match high-level operation descriptions in the test case. The last two columns provide the driver function calls that should be added after a match of a high-level operation description.

**Object diagram:**



**Test Case Generation:**

1. Identify input values  
 System.initialized = true  
 System.occupancyStatus.occupantClassForAirbagControl = Adult  
 System.occupancyStatus.occupantClassForSeatBeltReminder = Adult
2. Generate high-level operation descriptions  
 2.1 Process the UseCaseStar step of line 2: add the following line to the test case  

|       |                           |
|-------|---------------------------|
| Input | System.initialized = true |
|-------|---------------------------|

 2.2 Process the Input step of line 4: add the following line to the test case  

|       |                                                                                                                     |
|-------|---------------------------------------------------------------------------------------------------------------------|
| Input | System.occ...tus.occupantClassForAirbagControl = Adult<br>System.occ...tus.occupantClassForSeatBeltReminder = Adult |
|-------|---------------------------------------------------------------------------------------------------------------------|

 2.3 Process the Exit step of line 9: add the following line to the test case  

|       |                                                                                                                            |
|-------|----------------------------------------------------------------------------------------------------------------------------|
| Check | AirbagControlUnit.allInstances() -> forAll(a a.status = a.bodySense.occupancyStatus.occupantClassForAirbagControl) AND ... |
|-------|----------------------------------------------------------------------------------------------------------------------------|
3. Generate calls to driver functions  
 Lines 2, 4, and 6 in Table 4 are generated by processing the high-level operation descriptions according to the Mapping Table

**Figure 10: Activities performed by UMTG to generate the test case in Table 4**

For example, Line 1 of the test case in Table 4 matches the expression ‘*System.initialized = true*’ and thus leads to the generation of Line 2 of the test case.

UMTG generates test oracles by translating OCL postconditions into the scripting language of the testing framework. In our current implementation the translation is simply performed by means of the mapping table (see the third row in Table 5 that replaces the post condition with the operation *ReadAndCheckBus*). As a future work we plan to integrate more complex strategies to perform the translation.

**Table 4: A generated test case for *BodySense*<sup>TM</sup>**

| Line | Operation           | Inputs/Expectations                                                                             |
|------|---------------------|-------------------------------------------------------------------------------------------------|
| 1    | Input               | System.initialized = true                                                                       |
| 2    | ResetPower          | Time=INIT_TIME                                                                                  |
| 3    | Input               | occupantClassForAirbagControl = Adult<br>occupantClassForSeatBeltReminder = Adult               |
| 4    | SetBus              | Channel=RELAY Capacitance=85                                                                    |
| 5    | Check               | AirbagControlUnit.allInstances() → forAll(a <br>a.status = a.bodySense.occupancyStatus.occup... |
| 6    | ReadAnd<br>CheckBus | D0=OCCUPIED<br>D1=OCCUPIED                                                                      |

## 11. CASE STUDY

We applied UMTG to automatically generate test cases from the use cases that correspond to the main functionalities of *BodySense*.

Table 6 provides an overview of the case study. The first three columns report information about the use cases: the use case id (*ID*), the number of steps (*Steps*), and the number of use case flows (*Flows*). The numbers show that the behaviour of the system is not trivial since each use case include many steps (ranging from 25 to 59), and several alternative flows (ranging from 8 to 13).

The column *OCL* shows the number of constraints we specified by using OCL. The number of constraints defined for each use case is low, ranging from 5 to 12. The effort required for writing OCL depends on domain knowledge and expertise with OCL. At the beginning of the project, with little OCL expertise and domain knowledge, the main author of this paper took on average one hour to write each constraint because he needed several meetings with IEE engineers to understand the specifications. After few months

**Table 5: Mapping table for *BodySense*<sup>TM</sup> (excerpt)**

| Pattern to match<br>(Operation and Inputs) |                                                                                  | Result<br>(Operation and Inputs) |                                     |
|--------------------------------------------|----------------------------------------------------------------------------------|----------------------------------|-------------------------------------|
| Input                                      | *ClassForAirbagControl = Adult<br>*SeatBeltReminder=Adult                        | Set Bus                          | Channel = RELAY<br>Capacitance = 85 |
| Input                                      | System.initialized = true                                                        | Reset Power                      | Time=INIT.TIME                      |
| Check                                      | AirbagControlUnit.allInstances() -> forAll(a a.status = a.bodySense.occupancy... | ReadAnd<br>CheckBus              | D0=OCCUPIED<br>D1=OCCUPIED          |

of experience it only takes him a few minutes to write complex constraints for *BodySense*<sup>3</sup>.

The columns *Scenarios Covered*, *IEE* and *UMTG*, show the number of use case scenarios covered by the test cases manually written by IEE software engineers, based on domain expertise, and by UMTG. We manually verified that UMTG covers all the scenarios of the use cases under test. Table 6 shows that UMTG covers use case scenarios that are not covered by manually derived test cases. This happens mainly because test engineers tend not to test all the scenarios that can be derived from bounded or global alternative flows (a same bounded or global alternative flow may lead to multiple paths, i.e. scenarios, in the Use Case Test Model). Doing that type of path analysis manually is indeed difficult for an engineer. Another reason for this discrepancy is that RUCM is more precise than standard use case templates, thus leading to the identification of more scenarios.

The last two columns of Table 6 provide information about the scalability of the technique by showing the overall number of nodes and arcs of the Use Case Test Model generated for each use case (in presence of included use cases we report the overall amount of nodes in the merged UCTM). These columns show that the generated models are not trivial, in fact they include a number of nodes and arcs that range from 27 to 154 and from 30 to 182, respectively.

The automatic generation of a test case, for each use case scenario, approximately took 12 minutes on average, with a maximum execution time of 56 minutes, and a minimum execution time of one minute. Most of the computation time was spent on constraint solving. These numbers clearly indicate that the approach scales since test case generation can be run overnight and each test case could easily be generated in parallel on a multicore platform.

**Table 6: Results obtained with the case study**

| ID | Use Cases |       | OCL | Scenarios Covered |      | UCTM<br>Nodes,ArCs |
|----|-----------|-------|-----|-------------------|------|--------------------|
|    | Steps     | Flows |     | IEE               | UMTG |                    |
| 1  | 50        | 8     | 9   | 30                | 37   | 154,182            |
| 2  | 44        | 13    | 7   | 10                | 13   | 46, 55             |
| 3  | 35        | 8     | 8   | 8                 | 8    | 35, 40             |
| 4  | 59        | 11    | 12  | 24                | 28   | 119,140            |
| 5  | 30        | 8     | 5   | 4                 | 8    | 32, 38             |
| 6  | 25        | 6     | 12  | 4                 | 6    | 27, 30             |

**Threats to validity.** The main threat to the validity of our case study regards the generalizability of the results. To limit this threat we applied the technique on an industrial case study that includes nontrivial use cases representative of an application domain: sensor systems in the automotive industry. To limit threats to the internal validity of the

<sup>3</sup>Note that approaches based on explicit behavioural models imply more modelling effort since they require both the specification of constraints for guards and postconditions and the modelling of message passing or state transitions [6, 4].

case study we compared the generated test cases with the ones developed by the IEE software engineers to verify that the differences between them depend only on the systematic coverage of the scenarios implemented by UMTG and not on implementation errors.

## 12. RELATED WORK

This section groups and discusses related techniques according to how they capture behavioral information.

Several approaches require that system requirements are given in UML behavioral models such as activity diagrams [19, 20], statecharts [4, 24], and sequence diagrams [6, 21]. Nebut et al. [21] propose a use case driven test generation approach based on system sequence diagrams. Briand and Labiche [6] use both activity and sequence diagrams to generate system test cases. While sequential dependencies between use cases are extracted from an activity diagram, sequences in a use case are derived from system sequence diagrams. In contrast, UMTG only requires use case specifications be complemented by a domain model including OCL constraints. OCL constraints are also required by techniques relying on behavioural models to generate system test cases.

There are works generating UML behavioral models from NL requirements [11, 31, 30]. For example, Yue et al. [29, 31] generate UML state machines from RUCM use cases. One issue that prevents using such an approach for automated test generation is that software engineers have to correct and complete the complex, generated models. With our work instead we avoid engineers to deal with complex behavioural models. Frohlich and Link [11] show how use cases can be systematically transformed into UML state charts, from which test cases are derived. The approach proposed by Frohlich and Link has two drawbacks: (i) generated test sequences have to be edited and (ii) test data have to be manually provided. In contrast, UMTG not only generates sequences of function calls that do not need to be modified, but also generates test data for these functions.

Different approaches use NLP techniques to derive test cases from NL requirements. Most of these approaches either require additional behavioral modeling, e.g., state diagrams [22], labelled transition systems [17], activity diagrams [14] or require manual intervention by the testers for test generation, e.g. provision of input test data [10], or manual test derivation [5]. However UMTG does not require behavioral modeling and the executable test cases are automatically derived from RUCM use case specifications and domain model.

In a very recent work, developed concurrently with ours, Zhang et al. [32] generate test cases from RUCM use cases by identifying sequences of use case steps that satisfy branch and loop coverage. Test cases cannot be executed automatically because they do not include concrete test data. In contrast, UMTG introduces some extensions in RUCM which, combined with the use of OCL constraints on a domain model, enable the generation of executable test cases with concrete test data. Another recent approach that generates test cases without test data from NL requirements is that of Sarmiento et al. [25]. Text2Test [27] instead uses NLP to support engineers in revising their use cases, but do not include means to automatically generate test cases.

In some contexts, test cases can be simply expressed in terms of sequences of system events without any additional input data, e.g. parameter values. For instance the interac-

tion test case generation approach proposed by Figuerdo et al. processes use case specifications written in a custom use case format to derive sequences of system operations and events [9]. UMTG complements this approach by focussing on the automation of functional system testing, which requires the generation of parameter values to guarantee the coverage of all the use case scenarios.

Carvalho et al. [7] generate executable test cases for reactive systems, from requirements written according to a restricted grammar and dictionary. There are two main limitations: the underlying dictionary may change from project to project, while a restricted grammar may not be suitable to express the requirements of certain systems. UMTG does not impose a restricted dictionary for use cases, but simply relies on a use case format that can be used to express use cases of different kinds of systems.

Similar to our approach, Archetest [16] requires a domain model and a use case specification with invariants, guard and post conditions. Unfortunately, Archetest can generate only test data without test case scenarios.

Techniques that support Behaviour Driven Development automatically generate test cases from requirements specified using test scenarios [28]. Test scenarios are textual descriptions of specific software executions, in fact they include the concrete values that should be used to test the system. UMTG instead relies upon use case specifications, which are used to express requirements in terms of a more general description of the software behaviour.

## 13. CONCLUSION

In this paper, we introduce UMTG, an automated approach for system test case generation including test data and executable test cases. It is driven by use case specifications, augmented with a domain model. Our motivation is to achieve automated test case generation by largely relying on what is common practice to document requirements for communication purposes among stakeholders.

To enable the automatic identification of test scenarios and test inputs we combine Natural Language Processing (NLP) with constraint solving. To extract behavioural information from use case specifications by means of NLP we rely upon use case specifications expressed in a restricted form called RUCM. Since RUCM was not originally designed for test generation, we introduced some extensions such as new keywords and new restrictions on existing keywords. We employed OCL to refine guard, pre, and post conditions automatically identified by NLP. We designed an algorithm that builds path conditions that capture the constraints under which alternative flows are executed. The algorithm automatically identifies test inputs by solving such path conditions with the aid of an OCL constraint solver.

Our industrial case study shows that UMTG works well with use case specifications for an automotive sensor system. The time required for test case generation enables the entire process to run over night. Our experience indicates that the requirements modelling needed by UMTG is entirely feasible in an industrial context.

## 14. ACKNOWLEDGMENT

Supported by the Fonds National de la Recherche, Luxembourg (FNR/P10/03). We would like to thank Thierry Stephany and the whole IEE software engineers team for their support.

## 15. REFERENCES

- [1] IEE sensing solutions. <http://www.iee.lu>.
- [2] The Object Constraint Language (OCL). <http://www.omg.org/spec/OCL/>.
- [3] S. Ali, M. Zohaib Iqbal, A. Arcuri, and L. Briand. Generating test data from OCL constraints with search techniques. *IEEE Transactions on Software Engineering*, 39(10):1376–1402, Oct 2013.
- [4] A. Bandyopadhyay and S. Ghosh. Test input generation using UML sequence and state machines models. In *Proceedings of the 2nd International Conference on Software Testing Verification and Validation (ICST'09)*, pages 121–130, 2009.
- [5] A. Bertolino and S. Gnesi. Use case-based testing of product lines. *ACM SIGSOFT Software Engineering Notes*, 28(5):355–358, 2003.
- [6] L. C. Briand and Y. Labiche. A UML-based approach to system testing. *Software and Systems Modelling*, 1(1):10–42, 2002.
- [7] G. Carvalho, D. Falcão, F. Barros, A. Sampaio, A. Mota, L. Motta, and M. Blackburn. Test case generation from natural language requirements based on SCR specifications. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing (SAC'13)*, pages 1217–1222, 2013.
- [8] H. Cunningham, D. Maynard, K. Bontcheva, and V. Tablan. GATE: A Framework and Graphical Development Environment for Robust NLP Tools and Applications. In *Proceedings of the 40th Anniversary Meeting of the Association for Computational Linguistics (ACL'02)*, 2002.
- [9] A. L. L. de Figueiredo, W. L. Andrade, and P. D. L. Machado. Generating interaction test cases for mobile phone systems from use case specifications. *SIGSOFT Software Engineering Notes*, 31(6):1–10.
- [10] M. J. Escalona, J. J. Gutierrez, M. Mejías, G. Aragón, I. Ramos, J. Torres, and F. J. Domínguez. An overview on test generation from functional requirements. *Journal of System and Software*, 84(8):1379–1393, Aug. 2011.
- [11] P. Frohlich and J. Link. Automated test case generation from dynamic models. In *14th European Conference on Object Oriented Programming (ECOOP'00)*, pages 472–491, 2000.
- [12] J. Gutiérrez, M. J. Escalona, M. Mejías, and J. Torres. A case study for generating test cases from use cases. In *Proceedings of the 2nd International Conference on Research Challenges in Information Science (RCIS'08)*, pages 83–96, 2008.
- [13] M. J. Harrold, G. Rothermel, and S. Sinha. Computation of interprocedural control dependence. In *Proceedings of the 1998 ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA'98)*, pages 11–20, New York, NY, USA, 1998. ACM.
- [14] B. Hasling, H. Goetz, and K. Beetz. Model based testing of system requirements using UML use case models. In *Proceedings of the 1st international conference on Software Testing, Verification, and Validation (ICST'08)*, pages 367–376. IEEE, 2008.
- [15] ISO. ISO-26262: Road vehicles – functional safety.
- [16] M. Kaplan, T. Klinger, A. M. Paradkar, A. Sinha, C. Williams, and C. Yilmaz. Less is more: A minimalistic approach to UML model-based conformance test generation. In *1st International Conference on Software Testing, Verification, and Validation (ICST'08)*, pages 82–91, 2008.
- [17] M. Katara and A. Kervinen. Making model-based testing more agile: a use case driven approach. In *Hardware and Software, Verification and Testing*, pages 219–234. Springer, 2007.
- [18] C. Larman. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process*. Prentice Hall Professional, 2002.
- [19] W. Linzhang, Y. Jiesong, Y. Xiaofeng, H. Jun, L. Xuandong, and Z. Guoliang. Generating test cases from UML activity diagram based on gray-box method. In *Proceedings of the 11th Asia-Pacific Software Engineering Conference (APSEC'04)*, 2004.
- [20] A. Nayak and D. Samanta. Synthesis of test scenarios using UML activity diagrams. *Software and Systems Modelling*, 10:63–89, 2011.
- [21] C. Nebut, F. Fleurey, Y. Le-Traon, and J.-M. Jezequel. Automatic test generation: a use case driven approach. *IEEE Transactions on Software Engineering*, 32(3):140–155, March 2006.
- [22] M. Riebisch, I. Philippow, and M. Götz. UML-based statistical test case generation. In *Objects, Components, Architectures, Services, and Applications for a Networked World*, pages 394–411. Springer, 2003.
- [23] V. Rus, M. Lintean, R. Banjade, N. Niraula, and D. Stefanescu. SEMILAR: The semantic similarity toolkit. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2013.
- [24] J. Ryser and M. Glinz. A scenario-based approach to validating and testing software systems using statecharts. In *Proceedings of the 12th International Conference on Software and Systems Engineering and their Applications (ICSSEA'99)*, 1999.
- [25] E. Sarmiento, J. C. S. d. P. Leite, and E. Almentero. C&L: Generating model based test cases from natural language requirements descriptions. In *1st International Workshop on Requirements Engineering and Testing (RET'14)*, pages 32–38, 2014.
- [26] M. Shirole and R. Kumar. UML behavioral model based test case generation: a survey. *ACM SIGSOFT Software Engineering Notes*, 38(4):1–13, 2013.
- [27] A. Sinha, S. M. S. Jr., and A. Paradkar. Text2test: Automated inspection of natural language use cases. In *Proceedings of the 3rd International Conference on Software Testing, Verification and Validation (ICST'10)*, pages 155–164. IEEE Computer Society, 2010.
- [28] M. Soeken, R. Wille, and R. Drechsler. Assisted behavior driven development using natural language processing. In *Proceedings of the 5th International Conference on Objects, Models, Components, Patterns (TOOLS'12)*, pages 269–287. Springer-Verlag, 2012.
- [29] T. Yue, S. Ali, and L. Briand. Automated transition from use cases to UML state machines to support state-based testing. In *Proceedings of the 7th European*

- conference on Modelling foundations and applications (ECMFA'11)*, pages 115–131, 2011.
- [30] T. Yue, L. Briand, and Y. Labiche. An automated approach to transform use cases into activity diagrams. In *Proceedings of the 6th European conference on Modelling foundations and applications (ECMFA'10)*, pages 337–353, 2010.
- [31] T. Yue, L. C. Briand, and Y. Labiche. Facilitating the transition from use case models to analysis models: Approach and experiments. *ACM Transactions on Software Engineering and Methodology*, 22(1), 2013.
- [32] M. Zhang, T. Yue, S. Ali, H. Zhang, and J. Wu. A systematic approach to automatically derive test cases from use cases specified in restricted natural languages. In *Proceedings of the 8th International Conference on System Analysis and Modeling: Models and Reusability (SAM'14)*, pages 142–157, 2014.